Uniwersytet Łódzki
Wydział Matematyki i Informatyki

Statystyki opisowe i wizualizacja danych

Jakub Furmanek

A. Cel ćwiczenia

Celem ćwiczenia było zapoznanie się z podstawowymi metodami statystyki opisowej oraz technikami wizualizacji danych na przykładzie rzeczywistego zbioru danych charakteryzujących samochody (plik carbig.csv).

W ramach ćwiczenia realizowano następujące cele szczegółowe:

- przygotowanie danych do analizy: wczytanie, obsługa braków, dodanie losowego szumu,
- obliczenie miar położenia (średnie, mediana, moda, kwartyle),
- wyznaczenie miar rozrzutu (wariancja, odchylenie standardowe, współczynnik zmienności),
- ocena asymetrii i kurtozy rozkładów zmiennych,
- wizualizacja rozkładów za pomocą histogramów, wykresów pudełkowych i probplot,
- analiza współzależności między zmiennymi (korelacja, scatter matrix, parallel coordinates),
- standaryzacja danych metodą z-score,
- analiza zużycia paliwa w zależności od parametrów technicznych pojazdów.

B. Część teoretyczna (treść zadania)

Statystyki opisowe

Statystyki opisowe to narzędzia służące do syntetycznego przedstawienia danych liczbowych. Pozwalają one scharakteryzować rozkład zmiennej za pomocą niewielkiej liczby miar.

Miary położenia informują o typowym poziomie wartości cechy. Do najważniejszych należą:

- średnia arytmetyczna — suma wartości podzielona przez ich liczbę,
- mediana — wartość środkowa w uporządkowanym zbiorze danych,
- moda — wartość najczęściej występująca,
- kwartyle — wartości dzielące zbiór na cztery równe części (Q1, Q2=mediana, Q3).

Miary rozrzutu opisują stopień zróżnicowania danych:

- wariancja — średni kwadrat odchyleń od średniej,
- odchylenie standardowe — pierwiastek kwadratowy z wariancji,
- rozstęp międzykwartylowy (IQR) — różnica między trzecim a pierwszym kwartylem,

- współczynnik zmienności — stosunek odchylenia standardowego do średniej.

Współczynnik asymetrii (skewness) określa stopień i kierunek asymetrii rozkładu. Wartość dodatnia oznacza asymetrię prawostronną (długi ogon po prawej stronie), ujemna — lewostronną, a wartość bliska zeru — rozkład symetryczny.

Kurtoza Pearsona opisuje stopień wysmukłości rozkładu w porównaniu z rozkładem normalnym. Wartość $K > 3$ oznacza rozkład leptokurtyczny (wysmukły), $K < 3$ — platykurtyczny (spłaszczony), a $K \approx 3$ — mezokurtyczny (podobny do normalnego).

Współczynnik korelacji Pearsona mierzy siłę i kierunek zależności liniowej między dwiema zmiennymi. Przyjmuje wartości od -1 do 1, gdzie wartości bliskie ±1 oznaczają silną zależność liniową.

C. Raport z wykonania ćwiczenia

Poniżej przedstawiono szczegółowy raport z wykonania wszystkich punktów ćwiczenia laboratoryjnego. Dla każdego zadania zamieszczono kompletny kod źródłowy Python, wyniki obliczeń w postaci tabel i wykresów oraz komentarz interpretacyjny.

Zadanie 1. Wczytanie i modyfikacja zbioru danych

Zadanie polegało na wczytaniu pliku carbig.csv, ustawieniu ziarna generatora liczb losowych, uzupełnieniu braków danych oraz dodaniu losowego szumu do wybranych zmiennych numerycznych.

Kod źródłowy:

```
# Import modułów
%matplotlib inline

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from scipy import stats
from pandas.plotting import parallel_coordinates, scatter_matrix
from IPython.display import display

# Ustawienia czytelności
pd.set_option("display.width", 180)
pd.set_option("display.max_columns", 100)
pd.set_option("display.float_format", lambda x: f"{x:,.4f}")

plt.style.use("default")
```

```python
# Funkcje pomocnicze
def pop_var(a):
    a = np.asarray(a, dtype=float)
    m = np.mean(a)
    return np.mean((a - m) ** 2)

def pop_std(a):
    return np.sqrt(pop_var(a))

def mean_abs_dev(a):
    a = np.asarray(a, dtype=float)
    m = np.mean(a)
    return np.mean(np.abs(a - m))

def skew_coef(a):
    # As = (1/s^3) * (1/n) * sum((xi - xbar)^3)
    a = np.asarray(a, dtype=float)
    m = np.mean(a)
    s = pop_std(a)
    if s == 0:
        return np.nan
    return np.mean((a - m) ** 3) / (s ** 3)

def kurtosis_pearson(a):
    # Kurtoza Pearsona = m4 / (m2^2)
    a = np.asarray(a, dtype=float)
    m = np.mean(a)
    m2 = np.mean((a - m) ** 2)
    if m2 == 0:
        return np.nan
    m4 = np.mean((a - m) ** 4)
    return m4 / (m2 ** 2)

def safe_gmean(a):
    a = np.asarray(a, dtype=float)
    return np.nan if np.any(a <= 0) else stats.gmean(a)

def safe_hmean(a):
    a = np.asarray(a, dtype=float)
    return np.nan if np.any(a == 0) else stats.hmean(a)

def asymmetry_label(as_val, eps=0.05):
    if pd.isna(as_val):
        return "brak"
    if as_val > eps:
        return "prawostronna"
    if as_val < -eps:
        return "lewostronna"
    return "w przybliżeniu symetryczna"

def kurtosis_label(k_val, eps=0.05):
    if pd.isna(k_val):
        return "brak"
    if k_val > 3 + eps:
        return "leptokurtyczny"
    if k_val < 3 - eps:
        return "platykurtyczny"
    return "mezokurtyczny"

# Przygotowanie danych
nr_gr = 5.5
r_k = 2026
seed = int(nr_gr * r_k)
```

```
np.random.seed(seed)
data_raw = pd.read_csv("carbig.csv")

columns = ["Acceleration", "Displacement", "Horsepower", "Weight", "MPG"]

print("Seed:")
print(f"seed = int({nr_gr} * {r_k}) = {seed}")

print("\nBraki danych przed uzupełnieniem:")
display(data_raw[columns + ["Cylinders"]].isna().sum().to_frame("liczba_brakow"))

data = data_raw.copy()

# Uzupełnienie braków modą w wymaganych kolumnach
for col in columns:
    data[col] = data[col].fillna(data[col].mode().iloc[0])

data["Cylinders"] = data["Cylinders"].fillna(data["Cylinders"].mode().iloc[0])

# Typy danych
data[columns] = data[columns].astype(float)
data["Cylinders"] = data["Cylinders"].astype(int)

# Zachowujemy kopię po imputacji, ale przed szumem
data_clean = data.copy()

# Macierz x i dodanie szumu ±1%
x = data_clean[columns].to_numpy(dtype=float)
noise_multiplier = 1 + (np.random.rand(*x.shape) * 2 - 1) / 100
x = x * noise_multiplier

# Dane robocze do dalszych zadań
data = data_clean.copy()
data.loc[:, columns] = x

print("\nBraki danych po uzupełnieniu:")
display(data[columns + ["Cylinders"]].isna().sum().to_frame("liczba_brakow"))

print("\nPodgląd danych (po dodaniu szumu):")
display(data[columns + ["Cylinders"]].head())

print("\nInformacje o zmiennych:")
display(data[columns + ["Cylinders"]].dtypes.to_frame("typ"))

print("\nOpis zmiennych numerycznych (po dodaniu szumu):")
display(data[columns].describe().round(4))
```

Wyniki:

Ziarno generatora: seed = 11143

	liczba_brakow
Acceleration	0
Displacement	0
Horsepower	6
Weight	0
MPG	8

Cylinders	0

Tabela 1.1. Braki danych przed uzupełnieniem

	liczba_brakow
Acceleration	0
Displacement	0
Horsepower	0
Weight	0
MPG	0
Cylinders	0

Tabela 1.2. Braki danych po uzupełnieniu

	Acceleration	**Displacement**	**Horsepower**	**Weight**	**MPG**	**Cylinders**
0	12.0727	304.9191	131.1791	3495.6229	18.1262	8.0000
1	11.4111	347.5924	166.2348	3720.4353	15.1071	8.0000
2	10.9210	318.6717	149.8491	3416.1962	17.9170	8.0000
3	11.9742	301.3966	151.4959	3418.7382	15.9712	8.0000
4	10.5425	300.5197	139.4265	3423.9709	17.0769	8.0000

Tabela 1.3. Podgląd pierwszych 5 wierszy danych (po szumie)

	typ
Acceleration	float64
Displacement	float64
Horsepower	float64
Weight	float64
MPG	float64
Cylinders	int64

Tabela 1.4. Typy danych zmiennych

	Acceleration	**Displacement**	**Horsepower**	**Weight**	**MPG**
count	406.0000	406.0000	406.0000	406.0000	406.0000
mean	15.5156	194.8133	105.7706	2979.0201	23.3134
std	2.7980	104.8982	38.8512	846.8803	7.8869
min	8.0436	67.8288	45.9786	1607.5390	9.0578
25%	13.6874	104.4730	75.6284	2235.8640	16.9111
50%	15.4117	149.6828	94.6940	2826.5178	22.3584
75%	17.1532	300.3755	131.2570	3602.3647	29.0671

max	24.7537	455.8613	230.4536	5142.7789	46.3528

Tabela 1.5. Statystyki opisowe zmiennych numerycznych

Komentarz:

Ustawienie ziarna generatora liczb losowych (seed = 11143) zapewnia powtarzalność wyników przy każdym uruchomieniu skryptu. Przed przetworzeniem pliku carbig.csv wykryto braki danych w kolumnach Horsepower (6 braków) oraz MPG (8 braków). Braki uzupełniono wartością modalną (modą) danej kolumny, co jest prostą metodą imputacji odpowiednią dla zmiennych o charakterze ciągłym.

Do każdej wartości zmiennych numerycznych dodano losowy szum w zakresie ±1% w celu symulacji naturalnej zmienności pomiarowej. Operacja ta nie zmienia istotnie charakteru rozkładów, ale spełnia wymaganie zadania. Kolumnę Cylinders zachowano do późniejszych zadań (zadanie 8 i 12), jako zmienną kategoryczną określającą liczbę cylindrów silnika.

Zadanie 2. Miary średnie

Wyznaczenie wartości minimalnych, maksymalnych oraz wszystkich miar średnich dla każdej zmiennej numerycznej.

Kod źródłowy:

```
rows = []
for col in columns:
    a = data[col].to_numpy(dtype=float)
    a_base = data_clean[col].to_numpy(dtype=float)
    w = np.ones_like(a) / len(a)

    rows.append({
        "min": np.min(a),
        "max": np.max(a),
        "srednia_aryt": np.mean(a),
        "srednia_wazona_(rowne_wagi)": np.sum(w * a),
        "mediana": np.median(a),
        "moda_przed_szumem": pd.Series(a_base).mode().iloc[0],
        "srednia_geom": safe_gmean(a),
        "srednia_harm": safe_hmean(a),
    })

tab2 = pd.DataFrame(rows, index=columns).round(4)
display(tab2)
```

Wyniki:

Zmienna	min	max	srednia_aryt	srednia_wazona	mediana	moda_przed_szumem	srednia_geom	srednia_harm
Acceleration	8.0436	24.7537	15.5156	15.5156	15.4117	14.5000	15.2590	14.9933
Displacement	67.8288	455.8613	194.8133	194.8133	149.6828	97.0000	168.9717	148.0003
Horsepower	45.9786	230.4536	105.7706	105.7706	94.6940	150.0000	99.4279	93.8309
Weight	1607.5390	5142.7789	2979.0201	2979.0201	2826.5178	1985.0000	2863.6019	2755.3129
MPG	9.0578	46.3528	23.3134	23.3134	22.3584	13.0000	21.9976	20.7257

Tabela 2.1. Miary średnie dla zmiennych numerycznych

Komentarz:

Średnia arytmetyczna jest podstawową miarą tendencji centralnej i dla większości zmiennych jest zbliżona do mediany. Dla zmiennych o rozkładach prawostronnych (np. Displacement, Horsepower) średnia jest wyższa od mediany, co wskazuje na obecność wartości odstających w górnym zakresie.

Mediana jako wartość środkowa jest bardziej odporna na obserwacje skrajne niż średnia arytmetyczna. Średnia geometryczna i harmoniczna są zawsze mniejsze lub równe średniej arytmetycznej i znajdują zastosowanie w analizach dotyczących wskaźników i tempa zmian.

Modę wyznaczono dla danych przed dodaniem szumu (zmienna data_clean), ponieważ dla danych ciągłych z losowym zaburzeniem moda traci sens interpretacyjny — każda wartość może być unikalna.

Zadanie 3. Kwartyle, rozstęp międzykwartylowy i wykresy pudełkowe

Wyznaczenie rozstępu, kwartyli, rozstępu międzykwartylowego, odchylenia ćwiartkowego oraz typowego obszaru zmienności. Utworzenie wykresów pudełkowych.

Kod źródłowy:

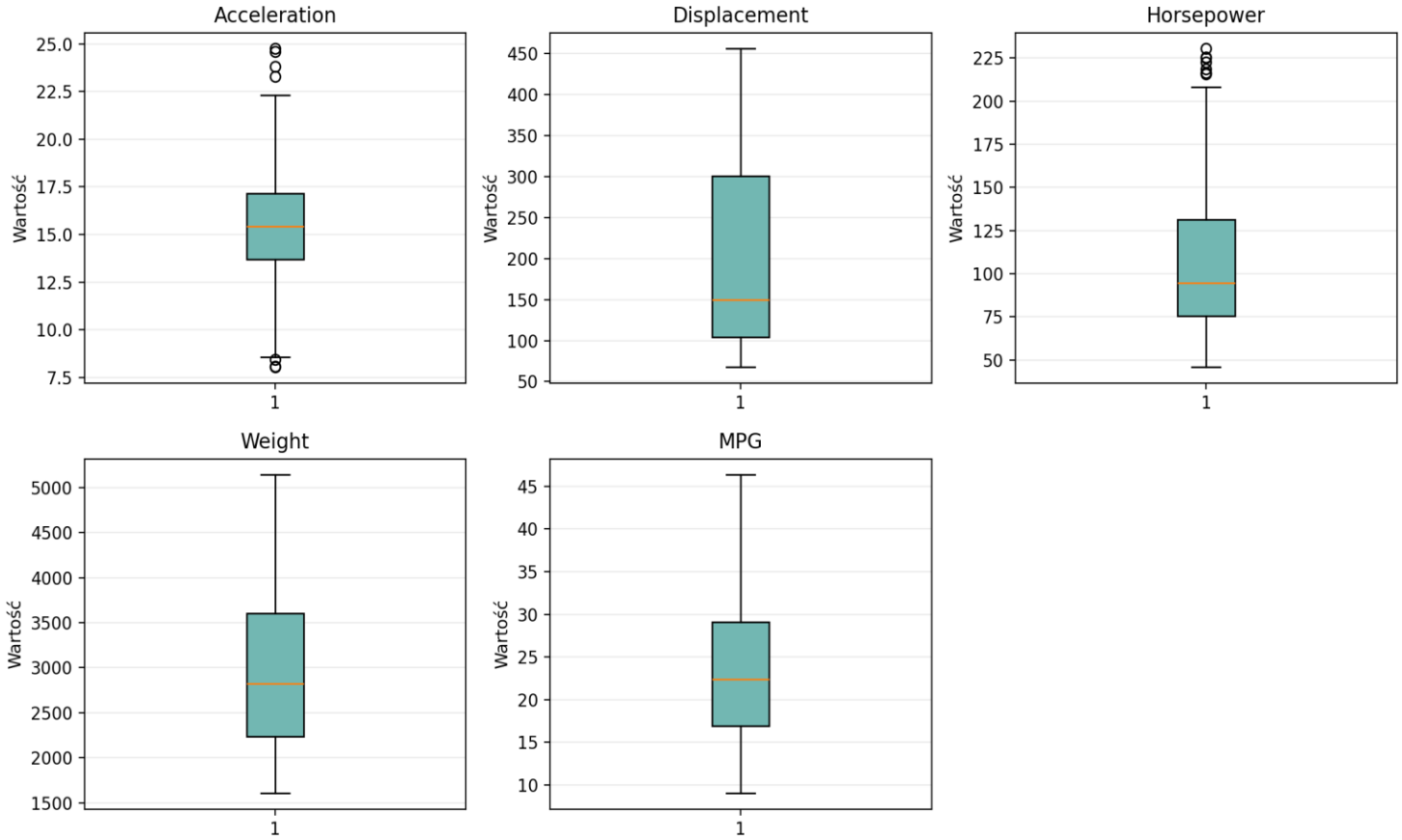
```
rows = []
for col in columns:
    a = data[col].to_numpy(dtype=float)
    q1, me, q3 = np.quantile(a, [0.25, 0.50, 0.75])
    iqr = q3 - q1
    Q = iqr / 2

    rows.append({
```

```
        "rozstep": np.max(a) - np.min(a),
        "Q1": q1,
        "Q2_mediana": me,
        "Q3": q3,
        "IQR": iqr,
        "odchylenie_cwiartkowe_Q": Q,
        "typowy_obszar_me_minus_Q": me - Q,
        "typowy_obszar_me_plus_Q": me + Q,
    })

tab3 = pd.DataFrame(rows, index=columns).round(4)
display(tab3)

# Boxploty osobno dla każdej zmiennej
fig, axes = plt.subplots(2, 3, figsize=(14, 8))
axes = axes.ravel()

for i, col in enumerate(columns):
    axes[i].boxplot(
        data[col].to_numpy(dtype=float),
        vert=True,
        patch_artist=True,
        boxprops=dict(facecolor="#8ecae6", edgecolor="black"),
        medianprops=dict(color="red", linewidth=2),
        whiskerprops=dict(color="black"),
        capprops=dict(color="black"),
        flierprops=dict(marker='o', markerfacecolor='orange', markersize=4,
markeredgecolor='black')
    )
    axes[i].set_title(col)
    axes[i].set_ylabel("Wartość")
    axes[i].grid(axis="y", alpha=0.25)

axes[-1].axis("off")

plt.tight_layout()
plt.show()
```

Wyniki:

Zmienna	rozstep	Q1	Q2_mediana	Q3	IQR	odchylenie_cwiartkowe_Q	typowy_obszar_me_minus_Q	typowy_obszar_me_plus_Q
Acceleration	16.7101	13.6874	15.4117	17.1532	3.4658	1.7329	13.6788	17.1447
Displacement	388.0325	104.4730	149.6828	300.3755	195.9025	97.9513	51.7315	247.6341
Horsepower	184.4750	75.6284	94.6940	131.2570	55.6287	27.8143	66.8797	122.5084
Weight	3535.2399	2235.8640	2826.5178	3602.3647	1366.5007	683.2504	2143.2674	3509.7682
MPG	37.2950	16.9111	22.3584	29.0671	12.1560	6.0780	16.2804	28.4364

Tabela 3.1. Kwartyle i miary rozproszenia pozycyjnego



Rys. 1. Wykresy pudełkowe dla analizowanych zmiennych

Komentarz:

Kwartyle dzielą uporządkowany zbiór danych na cztery równe części. Pierwszy kwartyl (Q1) to wartość poniżej której znajduje się 25% obserwacji, mediana (Q2) — 50%, a trzeci kwartyl (Q3) — 75%. Rozstęp międzykwartylowy (IQR) obejmuje środkowe 50% obserwacji i jest miarą rozproszenia odporną na wartości odstające.

Wykresy pudełkowe (boxploty) wizualizują rozkład danych poprzez przedstawienie mediany, kwartyli i wartości odstających. Ze względu na bardzo różne skale zmiennych (np. Weight osiąga wartości rzędu tysięcy, a Acceleration — dziesiątek), wykresy przedstawiono osobno dla każdej zmiennej, co znacząco poprawia czytelność i pozwala na dokładną ocenę rozkładu każdej cechy.

Zadanie 4. Miary rozrzutu

Wyznaczenie wszystkich miar rozrzutu dla każdej zmiennej numerycznej.

Kod źródłowy:

```
rows = []
for col in columns:
    a = data[col].to_numpy(dtype=float)
    xbar = np.mean(a)
    d = mean_abs_dev(a)
    s2 = pop_var(a)
    s = pop_std(a)

    q1, me, q3 = np.quantile(a, [0.25, 0.50, 0.75])
    Q = (q3 - q1) / 2

    rows.append({
        "odch_przecietne_d": d,
        "wariancja_s2": s2,
        "odch_std_s": s,
        "typowy_obszar_xbar_minus_s": xbar - s,
        "typowy_obszar_xbar_plus_s": xbar + s,
        "wsp_zmien_pos_Q_me": (Q / me) if me != 0 else np.nan,
        "wsp_zmien_d_xbar": (d / xbar) if xbar != 0 else np.nan,
        "wsp_zmien_s_xbar": (s / xbar) if xbar != 0 else np.nan,
    })

tab4 = pd.DataFrame(rows, index=columns).round(4)
display(tab4)
```

Wyniki:

Zmienna	odch_przecietne_d	wariancja_s2	odch_std_s	typowy_obszar_xbar_minus_s	typowy_obszar_xbar_plus_s	wsp_zmien_pos_Q_me	wsp_zmien_d_xbar	wsp_zmien_s_xbar
Acceleration	2.1763	7.8094	2.7945	12.7211	18.3102	0.1124	0.1403	0.1801
Displacement	92.2042	10976.5401	104.7690	90.0443	299.5822	0.6544	0.4733	0.5378
Horsepower	31.1669	1505.6983	38.8033	66.9672	144.5739	0.2937	0.2947	0.3669
Weight	719.5571	715439.7510	845.8367	2133.1834	3824.8568	0.2417	0.2415	0.2839
MPG	6.6060	62.0497	7.8772	15.4362	31.1905	0.2718	0.2834	0.3379

Tabela 4.1. Miary rozrzutu dla zmiennych numerycznych

Komentarz:

Wariancja i odchylenie standardowe są podstawowymi miarami rozproszenia danych wokół średniej. Odchylenie standardowe, wyrażone w tych samych jednostkach co dane pierwotne, jest szczególnie użyteczne w interpretacji.

Współczynnik zmienności (V = s/średnia) jest miarą względną, która umożliwia porównanie zmienności cech mierzonych w różnych jednostkach. Najniższą zmienność (18%) wykazuje zmienna Acceleration, a największą (54%) Displacement. Potwierdza to, że pojemność silnika jest cechą silnie różnicującą badane pojazdy.

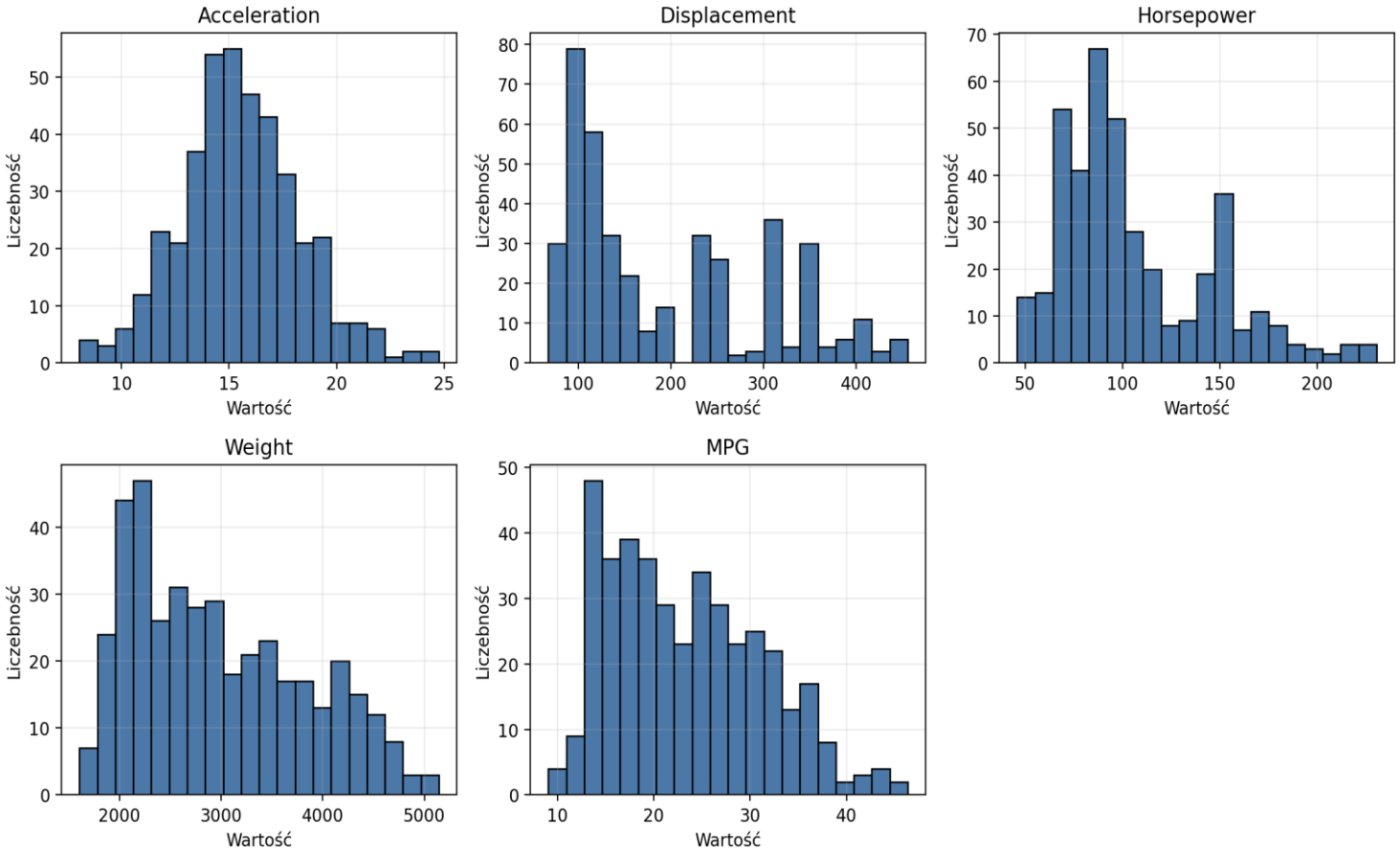
Zadanie 5. Histogramy

Utworzenie histogramów dla każdej zmiennej numerycznej.

Kod źródłowy:

```
fig, axes = plt.subplots(2, 3, figsize=(13, 7))
axes = axes.ravel()

for i, col in enumerate(columns):
    axes[i].hist(data[col].to_numpy(dtype=float), bins=20, edgecolor="black")
    axes[i].set_title(col)
    axes[i].set_xlabel("Wartość")
    axes[i].set_ylabel("Liczebność")
    axes[i].grid(alpha=0.25)

axes[-1].axis("off")
plt.tight_layout()
plt.show()
```

Wyniki:



Rys. 2. Histogramy analizowanych zmiennych numerycznych

Komentarz:

Histogramy pozwalają na graficzną ocenę kształtu rozkładu zmiennych. Większość analizowanych zmiennych wykazuje asymetrię prawostronną — dominują wartości niższe, a wartości wyższe (ogon rozkładu) występują rzadziej.

Zmienna MPG (mile per gallon) charakteryzuje się stosunkowo najbardziej regularnym rozkładem, choć także prawostronnym. Displacement i Horsepower mają wyraźnie prawostronne rozkłady z długim ogonem w kierunku wartości wyższych, co świadczy o obecności pojazdów o znacznie większej pojemności i mocy niż typowe dla zbioru.

Zadanie 6. Współczynniki asymetrii i kurtozy

Wyznaczenie współczynników asymetrii (skew) oraz kurtozy (kurtosis) dla każdej zmiennej numerycznej.

Kod źródłowy:

```
rows = []
for col in columns:
    a = data[col].to_numpy(dtype=float)
    As = skew_coef(a)
    K = kurtosis_pearson(a)

    rows.append({
        "skew_As": As,
        "asymetria": asymmetry_label(As),
        "kurtoza_Pearsona": K,
        "typ_kurtozy": kurtosis_label(K)
    })

tab6 = pd.DataFrame(rows, index=columns).round(4)
display(tab6)
```

Wyniki:

Zmienna	skew_As	asymetria	kurtoza_Pearsona	typ_kurtozy
Acceleration	0.2259	prawostronna	3.3619	leptokurtyczny
Displacement	0.6912	prawostronna	2.1825	platykurtyczny
Horsepower	0.9783	prawostronna	3.3896	leptokurtyczny
Weight	0.5041	prawostronna	2.1727	platykurtyczny
MPG	0.4695	prawostronna	2.4668	platykurtyczny

Tabela 6.1. Współczynniki asymetrii i kurtozy

Komentarz:

Wszystkie zmienne wykazują asymetrię prawostronną, co potwierdza obserwacje z histogramów. Najsilniejszą asymetrię wykazuje Horsepower (As = 0.98), co oznacza, że większość samochodów ma relatywnie niską moc silnika, podczas gdy niewielka liczba pojazdów ma bardzo wysoką moc.

Zmienne Acceleration i Horsepower charakteryzują się rozkładem leptokurtycznym (K > 3), co oznacza, że ich rozkłady są bardziej wysmukłe i mają cięższe ogony niż rozkład normalny — występują częściej wartości skrajne. Pozostałe zmienne mają rozkłady platykurtyczne (K < 3), czyli bardziej spłaszczone w porównaniu z rozkładem normalnym.

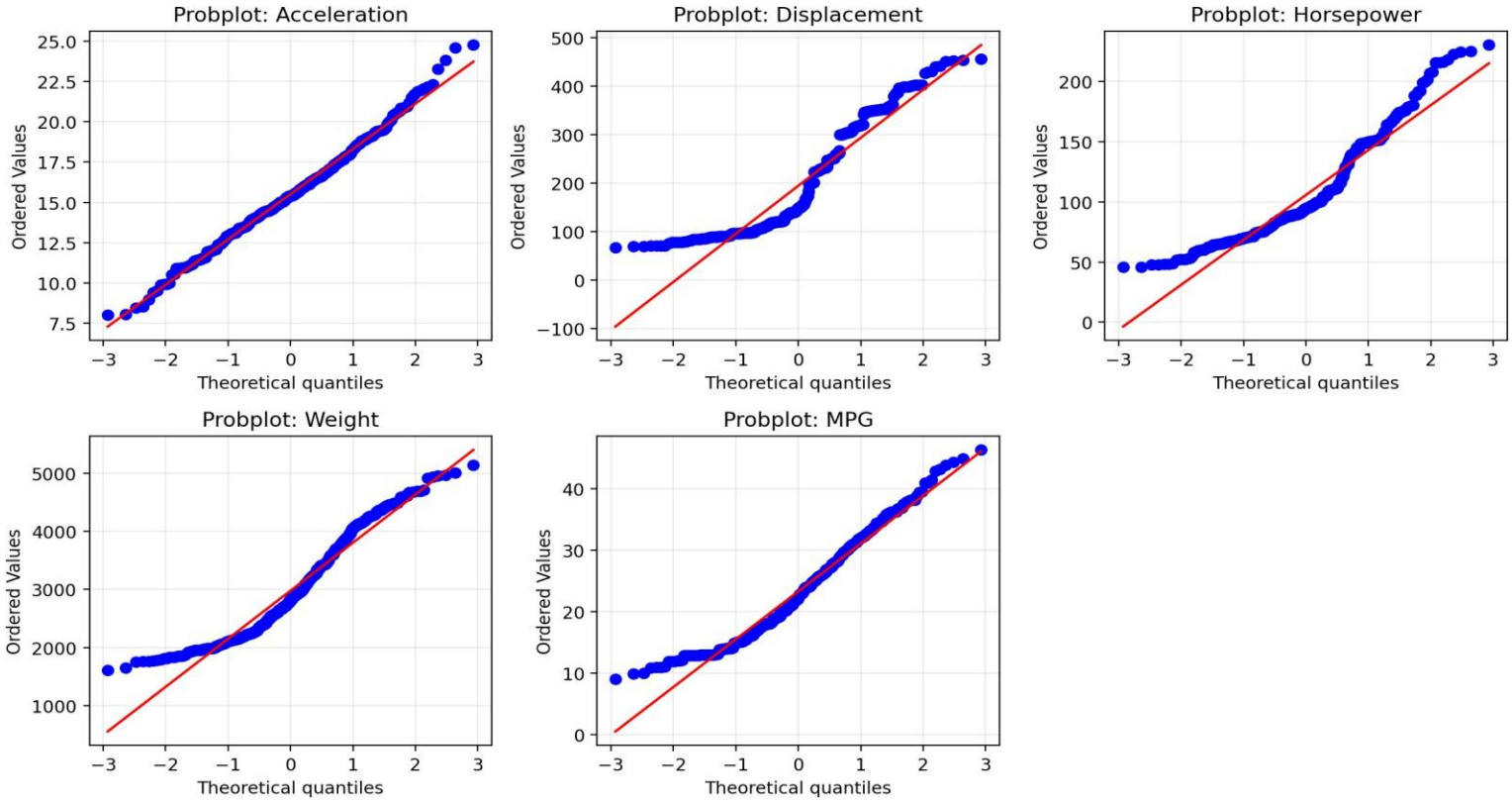
Zadanie 7. Wykresy probplot

Utworzenie wykresów probplot dla każdej zmiennej numerycznej. Wykres ten pozwala ocenić, jak bardzo rozkład empiryczny zmiennej odbiega od rozkładu normalnego.

Kod źródłowy:

```
fig, axes = plt.subplots(2, 3, figsize=(13, 7))
axes = axes.ravel()

for i, col in enumerate(columns):
    stats.probplot(data[col].to_numpy(dtype=float), dist="norm", plot=axes[i])
    axes[i].set_title(f"Probplot: {col}")
    axes[i].grid(alpha=0.25)

axes[-1].axis("off")
plt.tight_layout()
plt.show()
```

Wyniki:



Rys. 3. Wykresy probplot dla analizowanych zmiennych

Komentarz:

Wykres probplot (probability plot) porównuje kwantyle empiryczne badanej zmiennej z kwantylami teoretycznego rozkładu normalnego. Jeżeli punkty układają się blisko linii prostej, oznacza to, że rozkład zmiennej jest zbliżony do normalnego.

Wyraźne odchylenia od prostej, szczególnie na końcach wykresu, wskazują na asymetrię rozkładu, cięższe ogony lub obecność obserwacji odstających. W przypadku analizowanych zmiennych widoczne są odchylenia od normalności, co jest zgodne z wynikami współczynników asymetrii — większość zmiennych wykazuje prawostronne rozkłady odbiegające od rozkładu Gaussa.

Zadanie 8. Wykres parallel coordinates po standaryzacji

Utworzenie i interpretacja wykresu parallel coordinates. Przed wykonaniem wykresu dokonano standaryzacji danych metodą z-score, a następnie dodano kolumnę Cylinders jako zmienną klasową.

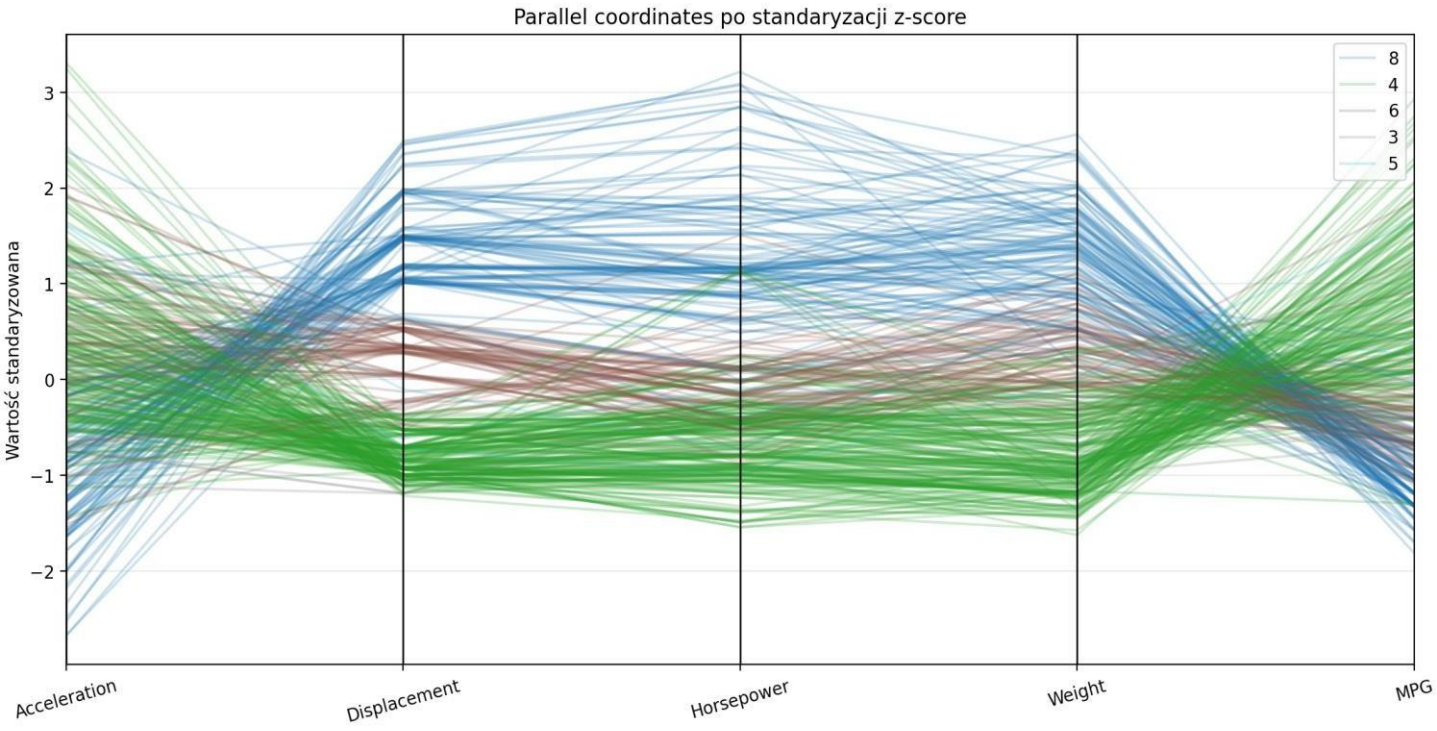
Kod źródłowy:

```
z = stats.zscore(data[columns].to_numpy(dtype=float), axis=0, ddof=0)
df_z = pd.DataFrame(z, columns=columns)
df_z["Cylinders"] = data["Cylinders"].astype(int).astype(str)

plt.figure(figsize=(13, 6))
parallel_coordinates(df_z, class_column="Cylinders", alpha=0.25,
colormap=plt.cm.tab10)
plt.title("Parallel coordinates po standaryzacji z-score")
plt.ylabel("Wartość standaryzowana")
plt.xticks(rotation=15)
plt.grid(alpha=0.25)
plt.show()

print(df_z.groupby("Cylinders")[columns].mean().round(3))
```

Wyniki:

Cylinders	Acceleration	Displacement	Horsepower	Weight	MPG
3	-0.8370	-1.1640	-0.1610	-0.6730	-0.3570
4	0.3910	-0.8100	-0.6580	-0.7880	0.7300
5	1.1260	-0.4710	-0.5970	0.1680	0.4980
6	0.2640	0.2220	-0.0960	0.2570	-0.4220
8	-0.9560	1.4360	1.3580	1.3320	-1.0720

Tabela 8.1. Średnie standaryzowane wartości według liczby cylindrów



Rys. 4. Wykres parallel coordinates po standaryzacji z-score

Komentarz:

Przed wykonaniem wykresu zastosowano standaryzację z-score (metoda zscore z scipy.stats), dzięki której wszystkie zmienne zostały przekształcone do średniej równej zero i odchylenia standardowego równego jeden. Pozwala to na porównywanie zmiennych mimo różnych jednostek i skal.

Kolumna Cylinders pełni rolę zmiennej klasowej (class_column) — każda linia na wykresie reprezentuje jeden samochód, a kolor linii odpowiada liczbie cylindrów w silniku. Wykres parallel coordinates pozwala zidentyfikować typowe profile pojazdów:

- Samochody z 4 cylindrami (kolor niebieski): niskie wartości Displacement, Horsepower i Weight, wysokie MPG — pojazdy ekonomiczne, lekkie i słabe.
- Samochody z 8 cylindrami (kolor pomarańczowy): wysokie Displacement, Horsepower i Weight, niskie MPG — pojazdy mocne, ciężkie, spalające dużo paliwa.

- Samochody z 6 cylindrami zajmują pozycję pośrednią.

Duże rozdzielenie linii między klasami wskazuje, że liczba cylindrów wyraźnie wiąże się z parametrami technicznymi i zużyciem paliwa pojazdu.

Zadanie 9. Macierz korelacji

Wyznaczenie współczynników korelacji liniowej (Pearsona) dla każdej pary zmiennych numerycznych.

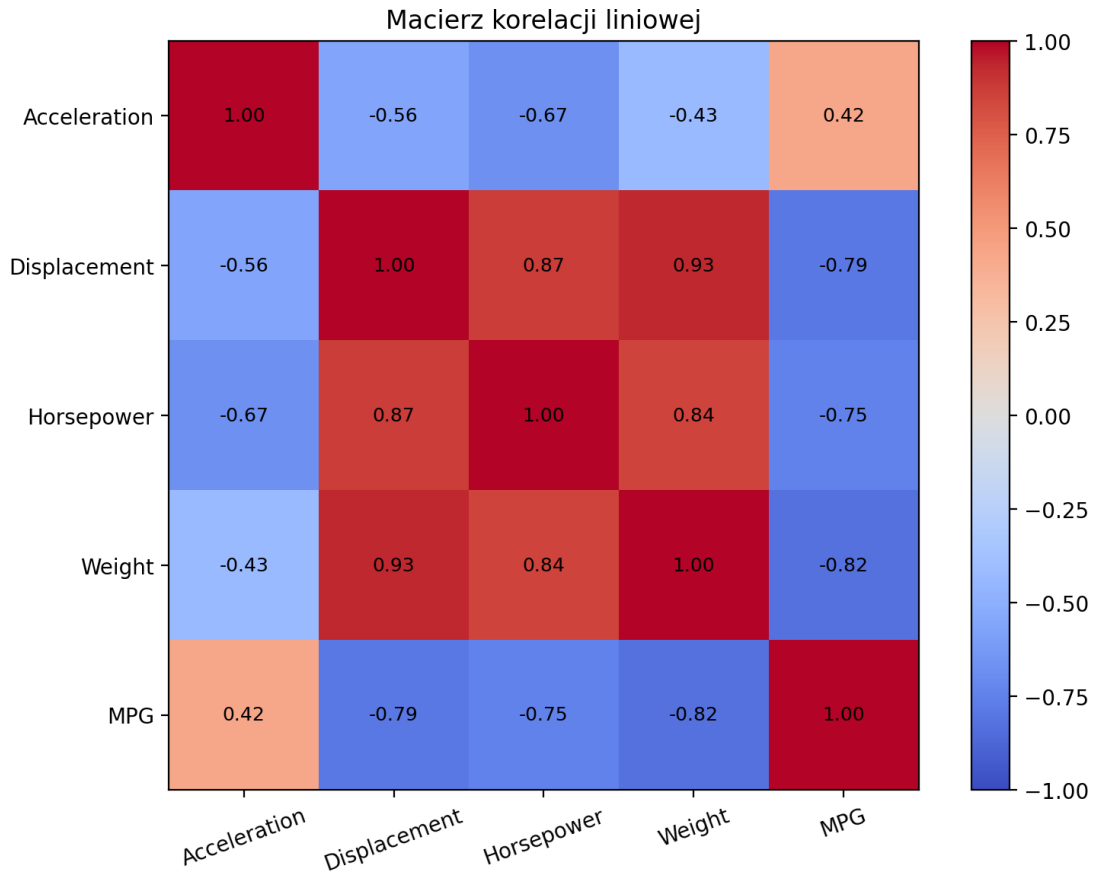
Kod źródłowy:

```
corr_matrix = data[columns].corr(method="pearson").round(4)
display(corr_matrix)

fig, ax = plt.subplots(figsize=(8, 6))
im = ax.imshow(corr_matrix, cmap="coolwarm", vmin=-1, vmax=1)

ax.set_xticks(range(len(columns)))
ax.set_yticks(range(len(columns)))
ax.set_xticklabels(columns, rotation=20)
ax.set_yticklabels(columns)

for i in range(len(columns)):
    for j in range(len(columns)):
        ax.text(j, i, f"{corr_matrix.iloc[i, j]:.2f}",
                ha="center", va="center", color="black")

plt.colorbar(im, ax=ax)
plt.title("Macierz korelacji liniowej")
plt.tight_layout()
plt.show()
```

Wyniki:

	Acceleration	**Displacement**	**Horsepower**	**Weight**	**MPG**
Acceleration	1.0000	-0.5571	-0.6745	-0.4294	0.4231
Displacement	-0.5571	1.0000	0.8746	0.9329	-0.7946
Horsepower	-0.6745	0.8746	1.0000	0.8441	-0.7474
Weight	-0.4294	0.9329	0.8441	1.0000	-0.8231
MPG	0.4231	-0.7946	-0.7474	-0.8231	1.0000

Tabela 9.1. Macierz korelacji liniowej Pearsona



Rys. 5. Mapa cieplna macierzy korelacji

Komentarz:

Współczynnik korelacji Pearsona mierzy siłę i kierunek zależności liniowej między zmiennymi. Przyjmuje wartości od -1 (silna korelacja ujemna) do +1 (silna korelacja dodatnia). Wartość bliska 0 oznacza brak zależności liniowej.

Najsilniejsze dodatnie korelacje:

- Displacement vs Weight (r = 0.93) — pojazdy o większej pojemności silnika są cięższe,
- Displacement vs Horsepower (r = 0.87) — większa pojemność wiąże się z większą mocą,
- Horsepower vs Weight (r = 0.84) — mocniejsze samochody są cięższe.

Najsilniejsze ujemne korelacje:

- Weight vs MPG (r = -0.82) — cięższe pojazdy spalają więcej paliwa,
- Displacement vs MPG (r = -0.79) — większa pojemność silnika = większe spalanie,
- Horsepower vs MPG (r = -0.75) — mocniejsze silniki spalają więcej.

Zmienna Acceleration ma słabszą korelację z pozostałymi cechami, choć ujemna korelacja z Horsepower (r = -0.67) wskazuje, że pojazdy o większej mocy osiągają lepsze przyspieszenie (niższy czas 0-60 mph).

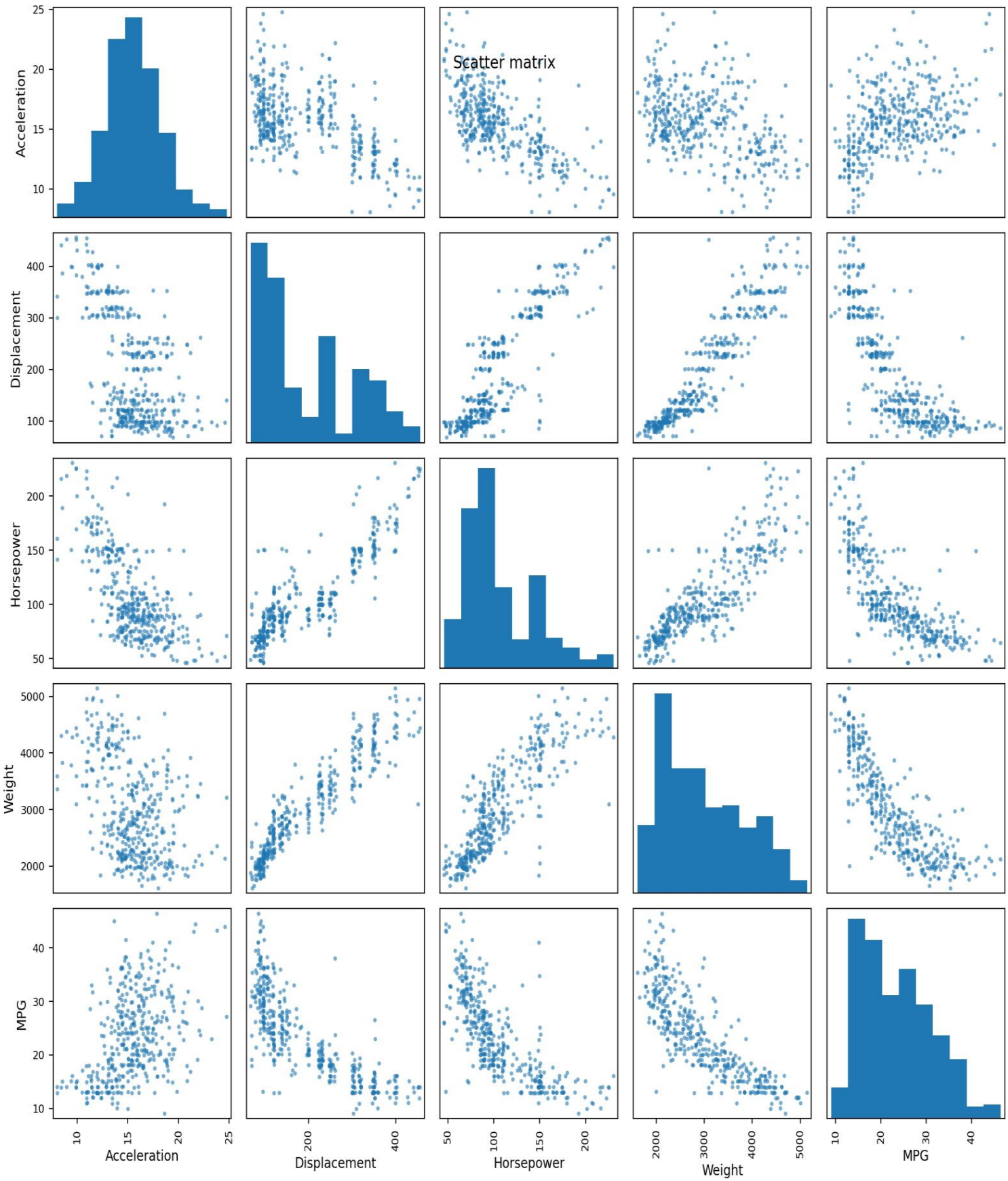
Zadanie 10. Scatter matrix

Utworzenie wykresu scatter matrix obrazującego współzależności między każdą parą zmiennych numerycznych.

Kod źródłowy:

```
scatter_matrix(data[columns], figsize=(12, 12), diagonal="hist",
                alpha=0.6, grid=True)
plt.suptitle("Scatter matrix", y=0.92)
plt.show()
```

Wyniki:


Scatter matrix
Acceleration
Displacement
Horsepower
Weight
MPG

Rys. 6. Macierz wykresów rozproszenia (scatter matrix)

Komentarz:

Macierz wykresów rozproszenia (scatter matrix) wizualizuje relacje między każdą parą zmiennych. Na przekątnej wykreślono histogramy rozkładów poszczególnych zmiennych, a poza przekątną - wykresy rozproszenia pokazujące zależności dwuwymiarowe.

Wykresy punktowe potwierdzają obserwacje z macierzy korelacji:

- Wyraźne dodatnie zależności liniowe: Displacement-Horsepower, Displacement-Weight, Horsepower-Weight,
- Wyraźne ujemne zależności liniowe: Weight-MPG, Horsepower-MPG, Displacement-MPG,
- Zależności są w większości liniowe lub zbliżone do liniowych.

Scatter matrix jest użytecznym narzędziem do szybkiego przeglądu struktury wielowymiarowego zbioru danych i identyfikacji potencjalnych zależności przed bardziej zaawansowaną analizą.

Zadanie 11. Zużycie paliwa

Wyznaczenie dodatkowej zmiennej Zuzycie reprezentującej zużycie paliwa w litrach na 100 km na podstawie zmiennej MPG. Sporządzenie wykresów zależności zużycia paliwa od pozostałych zmiennych.

Kod źródłowy:

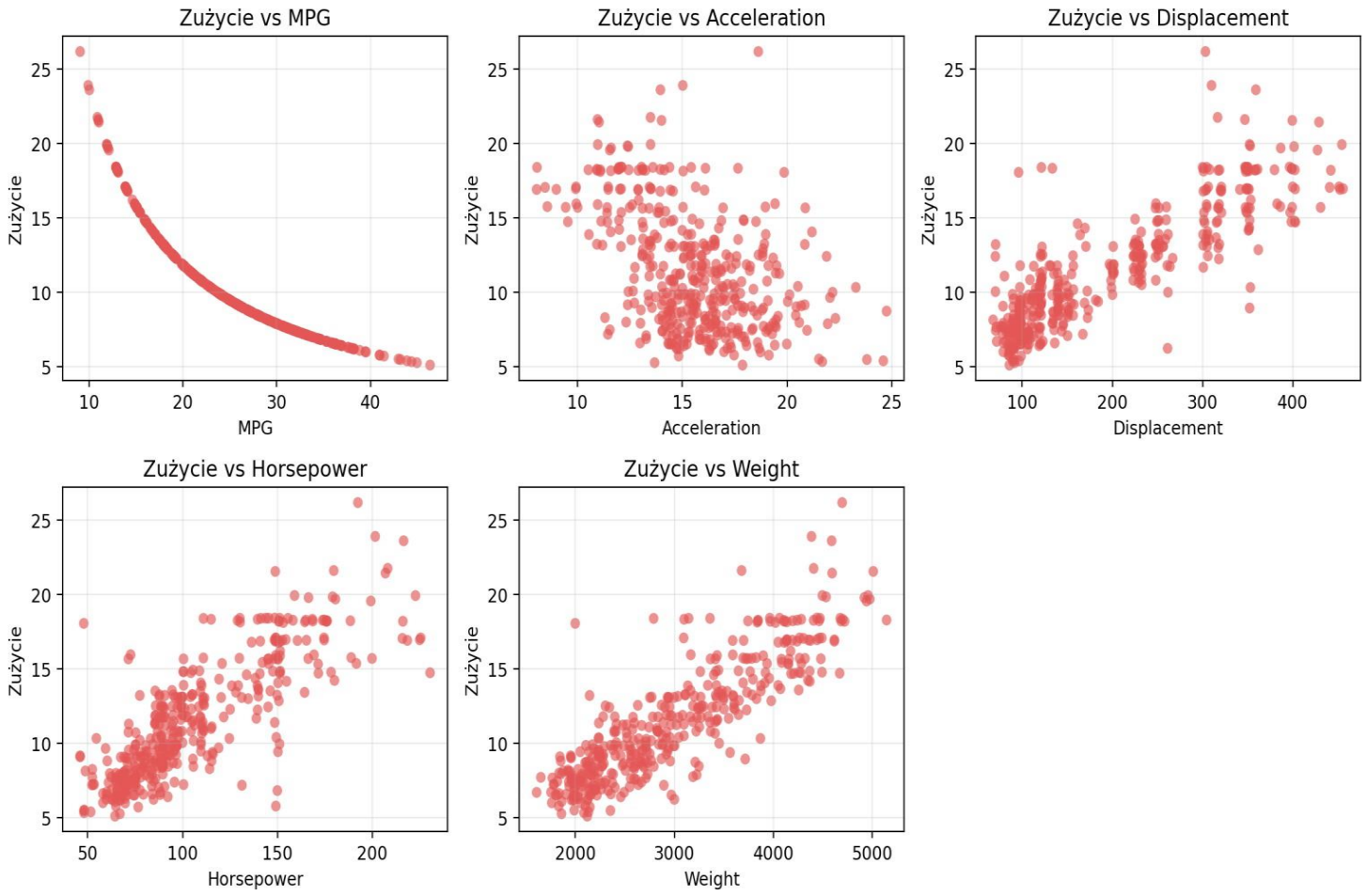
```
# Zużycie paliwa w l/100 km
# 1 mila = 1.6 km, 1 galon = 3.79 litra
data["Zuzycie"] = (3.79 * 100) / (1.6 * data["MPG"])

print(data[["MPG", "Zuzycie"]].head())

fig, axes = plt.subplots(2, 3, figsize=(14, 8))
axes = axes.ravel()

pairs = ["MPG", "Acceleration", "Displacement", "Horsepower", "Weight"]

for ax, col in zip(axes, pairs):
    ax.scatter(data[col], data["Zuzycie"], alpha=0.65)
    ax.set_xlabel(col)
    ax.set_ylabel("Zuzycie [l/100 km]")
    ax.set_title(f"Zuzycie vs {col}")
    ax.grid(alpha=0.25)

axes[-1].axis("off")
```

```
plt.tight_layout()
plt.show()
```

Wyniki:

	MPG	Zuzycie
0	18.1262	13.0681
1	15.1071	15.6797
2	17.9170	13.2207
3	15.9712	14.8314
4	17.0769	13.8711

Tabela 11.1. Przykładowe wartości MPG i Zużycia



Rys. 7. Zależność zużycia paliwa od wybranych zmiennych

Komentarz:

Zmienna Zużycie (zużycie paliwa w litrach na 100 km) została obliczona na podstawie MPG zgodnie z przelicznikiem jednostek:

Zużycie [l/100km] = (3.79 × 100) / (1.6 × MPG)

Analizując wykresy rozproszenia:

- Zużycie vs MPG: Silna ujemna zależność nieliniowa (hiperboliczna) — im wyższe MPG (więcej mil na galon), tym niższe zużycie w l/100 km. To bezpośredni wynik zastosowanego przelicznika.
- Zużycie vs Displacement/Horsepower/Weight: Dodatnie zależności — większa pojemność, moc i masa pojazdu wiążą się z wyższym zużyciem paliwa. Zależności te są w dużej mierze liniowe.
- Zużycie vs Acceleration: Słaba dodatnia zależność — pojazdy o lepszym przyspieszeniu (niższy czas) mają nieznacznie niższe spalanie.

Zadanie 12. Średnie zużycie paliwa według liczby cylindrów

Obliczenie średniego zużycia paliwa dla samochodów z 3, 4, 5, 6 i 8 cylindrami.

Kod źródłowy:

```
mean_consumption = (
    data[data["Cylinders"].isin([3, 4, 5, 6, 8])]
    .groupby("Cylinders")["Zuzycie"]
    .mean()
    .reindex([3, 4, 5, 6, 8])
    .to_frame("srednie_Zuzycie_l_na_100_km")
    .round(4)
)

print(mean_consumption)
```

Wyniki:

Cylinders	srednie_Zuzycie_l_na_100_km
3	11.6821
4	8.5309
5	9.2228
6	12.1899
8	16.4440

Tabela 12.1. Średnie zużycie paliwa według liczby cylindrów

Komentarz:

Średnie zużycie paliwa wyraźnie rośnie wraz z liczbą cylindrów. Samochody 4-cylindrowe są najbardziej ekonomiczne (8.53 l/100 km), podczas gdy pojazdy 8-cylindrowe spalają niemal dwukrotnie więcej (16.44 l/100 km).

Niewielką anomalią są samochody 5-cylindrowe, które mają nieco niższe spalanie (9.22 l/100 km) niż można by oczekiwać w porównaniu z 6-cylindrowymi (12.19 l/100 km). Może to wynikać z ich specyfiki konstrukcyjnej oraz niewielkiej liczebności w zbiorze danych.

Wyniki te są zgodne z intuicją techniczną: większa liczba cylindrów wiąże się z większą pojemnością i mocą silnika, a tym samym wyższym zużyciem paliwa. Zależność ta została wcześniej potwierdzona przez wysokie korelacje między Cylinders a Displacement, Horsepower, Weight oraz ujemną korelację z MPG.

D. Wnioski końcowe

1. Wszystkie analizowane zmienne numeryczne wykazują asymetrię prawostronną i rozkłady odbiegające od rozkładu normalnego. Najsilniejszą asymetrię zaobserwowano dla zmiennej Horsepower (As = 0.98).

2. Parametry techniczne pojazdów (pojemność, moc, masa) są silnie skorelowane ze sobą. Największą korelację zaobserwowano między Displacement a Weight (r = 0.93), co jest zgodne z zasadami konstrukcji pojazdów mechanicznych.

3. Zużycie paliwa jest silnie powiązane z parametrami technicznymi pojazdu. Istnieje wyraźna ujemna zależność między MPG a zużyciem paliwa wyrażonym w l/100 km (zależność hiperboliczna wynikająca z przelicznika jednostek). Zużycie rośnie wraz ze wzrostem masy (r = -0.82 dla Weight vs MPG), mocy i pojemności silnika.

4. Liczba cylindrów jest kluczowym czynnikiem wpływającym na parametry techniczne i ekonomiczne pojazdu. Samochody 8-cylindrowe charakteryzują się wysoką mocą, dużą masą i znacznym spalaniem (16.44 l/100 km), podczas gdy pojazdy 4-cylindrowe są lżejsze, słabsze i zdecydowanie bardziej ekonomiczne (8.53 l/100 km).

5. Standaryzacja danych metodą z-score pozwoliła na porównanie zmiennych o różnych skalach na wykresie parallel coordinates i ujawniła wyraźne różnice w profilach technicznych pojazdów o różnej liczbie cylindrów. Wykres ten stanowi efektywne narzędzie do wizualizacji wielowymiarowych zależności.

6. Zastosowane miary położenia i rozrzutu (średnia, mediana, odchylenie standardowe, IQR) pozwoliły na kompleksową charakterystykę rozkładów zmiennych. Współczynnik zmienności wykazał największą zmienność dla zmiennej Displacement (54%), co potwierdza jej wysoką siłę dyskryminacyjną.

7. Wizualizacje (histogramy, boxploty, wykresy rozproszenia, scatter matrix) potwierdziły wyniki obliczeń liczbowych i znacząco ułatwiły interpretację struktury danych. Scatter matrix okazała się szczególnie użyteczna do szybkiego przeglądu zależności wielowymiarowych.

8. Metoda uzupełniania braków danych modą okazała się wystarczająca dla niniejszej analizy opisowej. W przypadku bardziej zaawansowanych analiz predykcyjnych lub modelowania warto rozważyć inne metody imputacji (np. regresja, k-NN, wielokrotna imputacja).

9. Dodanie losowego szumu ±1% do wartości zmiennych numerycznych nie zmieniło istotnie charakteru ich rozkładów, co potwierdza odporność miar statystycznych na niewielkie zaburzenia losowe. Operacja ta symuluje naturalną zmienność pomiarową występującą w rzeczywistych zbiorach danych.

10. Analiza statystyczna i wizualizacja danych pozwoliły na pełne zrozumienie struktury badanego zbioru oraz zidentyfikowanie kluczowych zależności między cechami technicznymi pojazdów. Wyniki te mogą stanowić podstawę do dalszych analiz predykcyjnych i modelowania.

Literatura

Materiały wykładowe i ćwiczeniowe do przedmiotu Zaawansowane Metody Eksploracji, Prof. dr hab. inż. Grzegorz Dudek, Wydział Matematyki i Informatyki, Uniwersytet Łódzki, 2026.